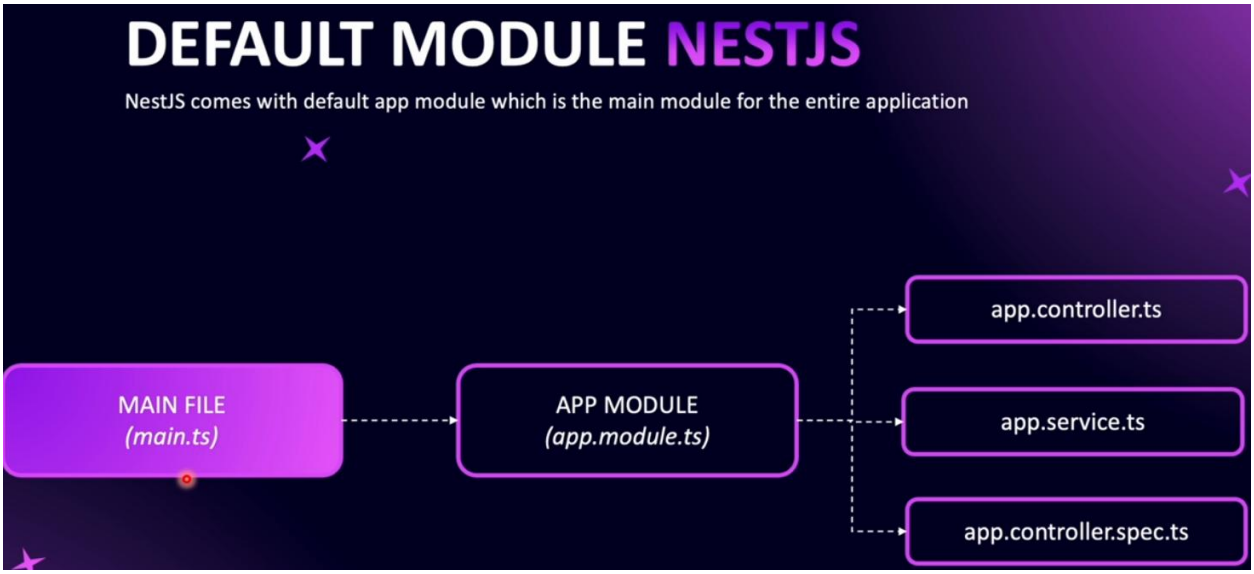
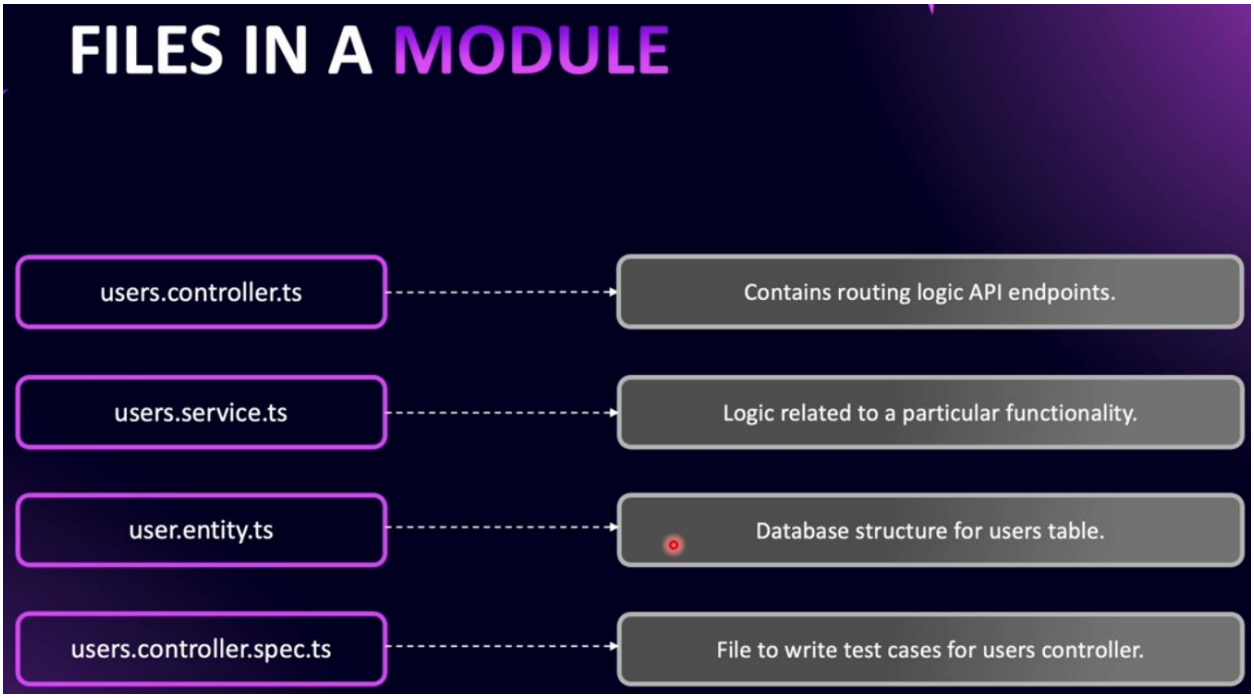
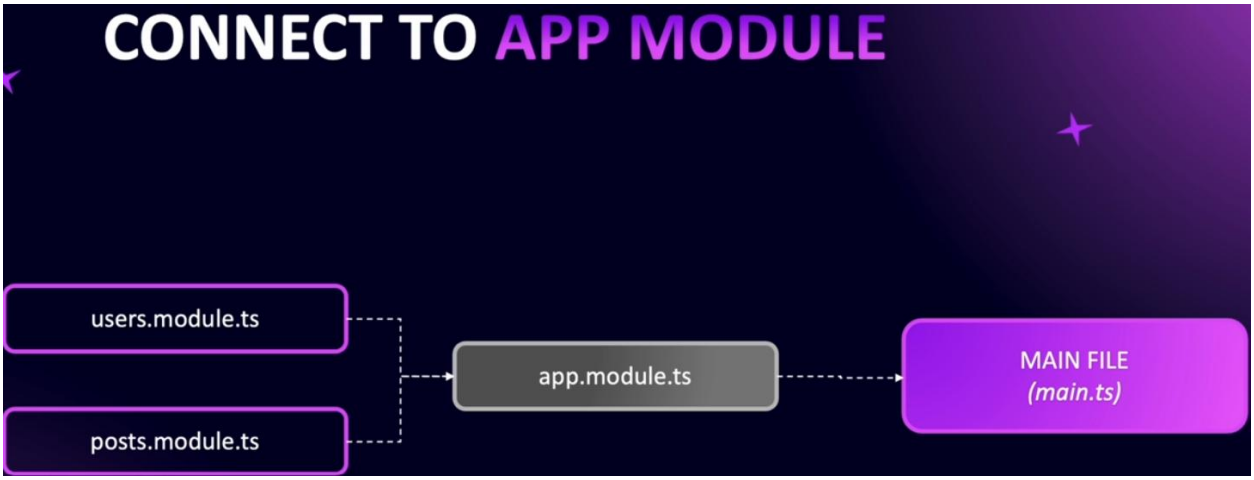

DEFAULT MODULE NESTJS
NestJS comes with default app module which is the main module for the entire application
MAIN FILE
(main.ts)
APP MODULE
(app.module.ts)
app.controller.ts
app.service.ts
app.controller.spec.ts

**********


FILES IN A MODULE
users.controller.ts
Contains routing logic API endpoints.
users.service.ts
Logic related to a particular functionality.
user.entity.ts
Database structure for users table.
users.controller.spec.ts
File to write test cases for users controller.

**********




**********

To generate new module: *nest generate module ModuleName*

**********

To generate new controller: *nest generate controller ControllerName*

**********

```
import { Controller, Get, Param } from '@nestjs/common';

@Controller('users')
export class UsersController {
  @Get('/')
  public getUsers() {
    return [
      { name: 'JLoka-01', job: 'Web Developer' },
      { name: 'JLoka-02', job: 'Mobile App Developer' },
    ];
  }

  // Note: If we don't set 'id' with param then id will become an object
  @Get('/:id')
  public getUserById(@Param('id') id: number) {
    // console.log(`The id is: ${id}`);
    return { id: id, name: 'JLoka-01', job: 'ITE Developer' };
  }
@Post('/')
  public createNewUser(@Body() data, @Headers() headers) {
    console.log(`The data is: ${JSON.stringify(data)}`);
    console.log('headers are: ', headers);
    return { msg: 'User Created Successfully' };
  }

}
      ********************************************************************
```

To add pipe to request:

Note: When we use pip for params they must be required

```
import {
  Body, Controller, Get, Headers, Param,
  Post, Query, ParseIntPipe,
} from '@nestjs/common';

  // Note: If we don't set 'id' with param then id will become an object
  @Get('/:id')
  public getUserById(
    @Param('id', ParseIntPipe) id: number,
    @Query('sort') sort: string,
  ) {
    // console.log(`The id is: ${id}`);
    return {
      type_: typeof id,
      id: id,
      name: 'JLoka-01',
      job: 'ITE Developer',
      sort,
    };
  }
```

**********

By using global pipes for validation we can't avoid the problem of extra values in the request, so the attacker can pass malicious content to our request

**********

To use validation with pipe: *npm i class-transformer*

```
import { ValidationPipe } from '@nestjs/common';
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
    }),
  );

  await app.listen(process.env.PORT ?? 3000);
}
```

**********

```
app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );
      ------------------------------------------------------

  @Post('/')
  public createNewUser(@Body() userDto: CreateUserDto) {
    console.log(`The data is: ${JSON.stringify(userDto)}`);
    console.log(typeof userDto); // Object
    console.log(userDto instanceof CreateUserDto);
// true, but without transformer: true in global pipe, it will be: false
    return { msg: 'User Created Successfully' };
  }
      ********************************************************************
```

To validate any number type (else we get bad request):

```
import { Type } from 'class-transformer';
import { IsInt } from 'class-validator';

export class GetUsersParamDto {
    @IsInt()
    @Type(() => Number)
    id?: number;
}
      ---------------------------------------------

  @Get('/:id')
  public getUserById(@Param() t1: GetUsersParamDto) {
    return {
      type_: typeof t1,
      id: t1,
      name: 'JLoka-01',
      job: 'ITE Developer',
    };
  }
      ---------------------------------------------
```

```
// To avoid the parsing errors of ts
  @Get('/')
  public getUsers() {
    return {
      name: 'JLoka-01',
      job: 'ITE Developer',
    };
  }
```

**********

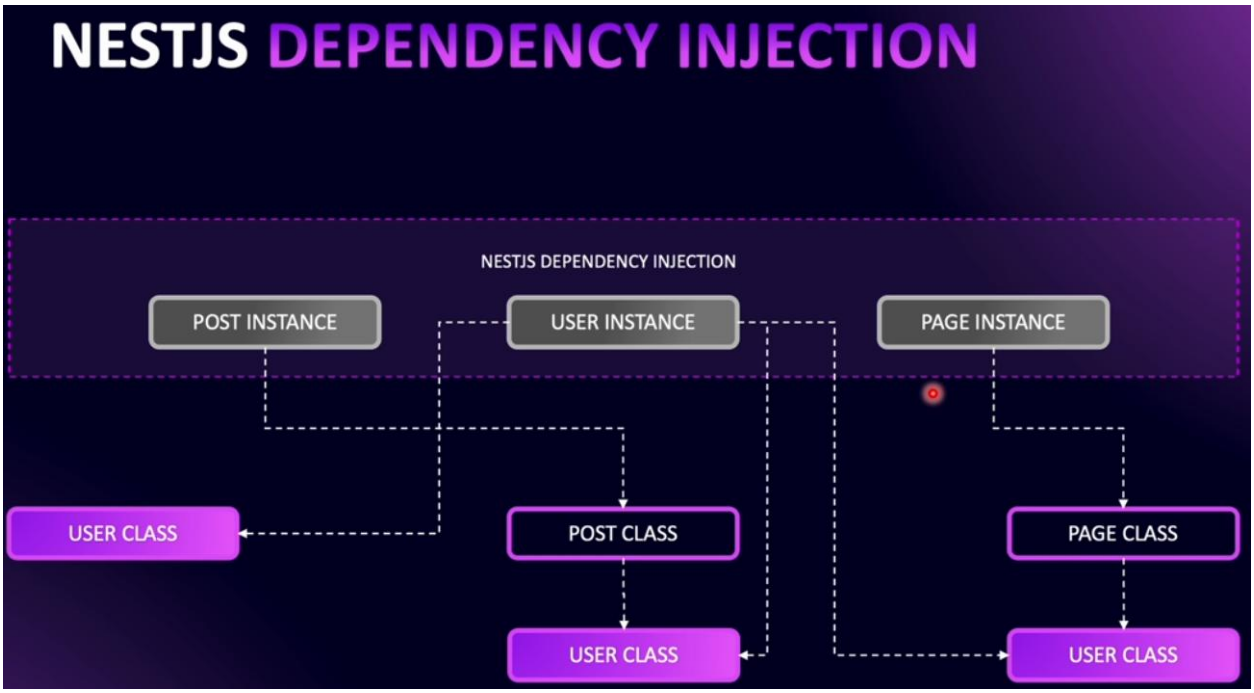
To install mapped types to avoid code duplication: npm i @nestjs/mapped-types

And to use it, we can use extends:

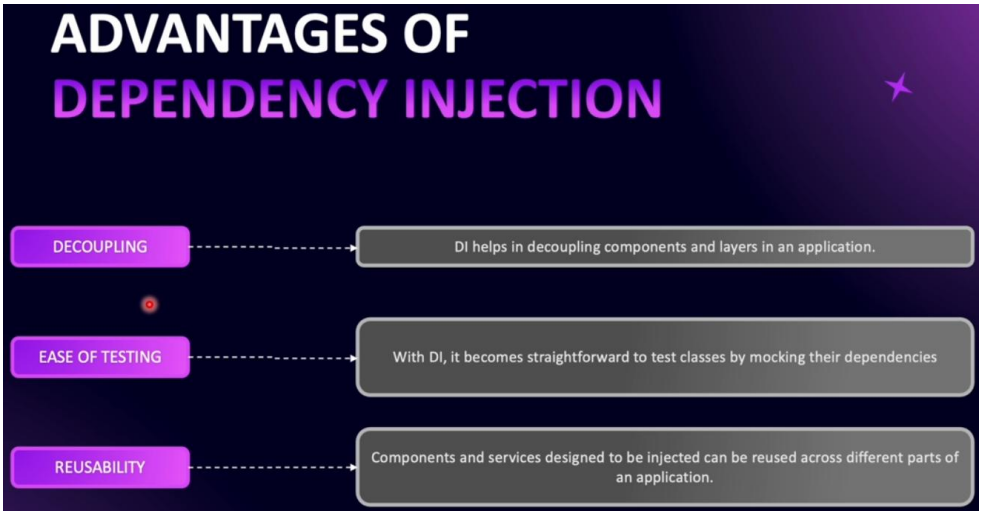
```
import { PartialType } from '@nestjs/mapped-types';
import { CreateUserDto } from './create-user.dto';

// Make all properties of CreateUserDto are optional
export class UpdateUserPartDTO extends PartialType(CreateUserDto) {}
```

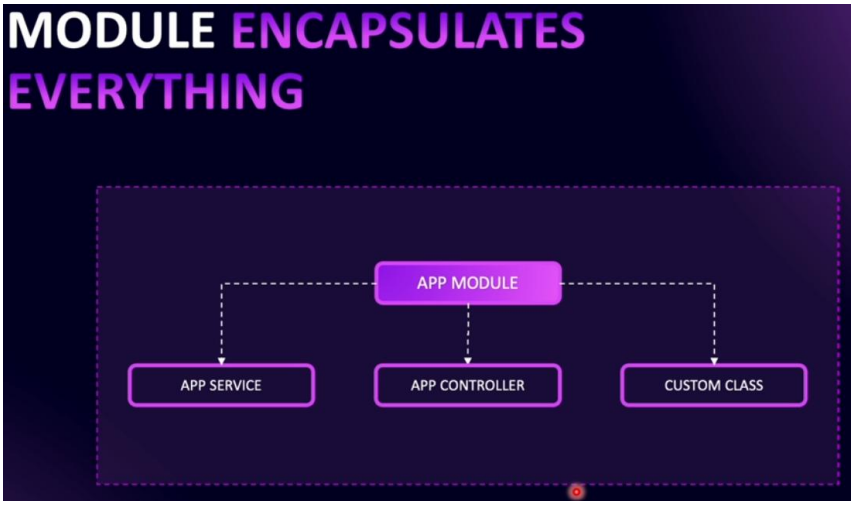
**********



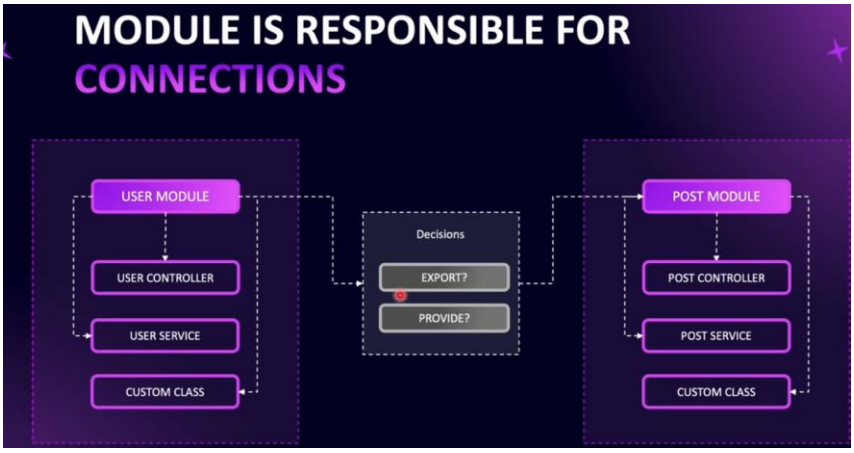

**********


ADVANTAGES OF
DEPENDENCY INJECTION
DECOUPLING
DI helps in decoupling components and layers in an application.
EASE OF TESTING
With DI, it becomes straightforward to test classes by mocking their dependencies
REUSABILITY
Components and services designed to be injected can be reused across different parts of an application.

**********


MODULE ENCAPSULATES
EVERYTHING
APP MODULE
APP SERVICE
APP CONTROLLER
CUSTOM CLASS

**********


MODULE IS RESPONSIBLE FOR
CONNECTIONS
USER MODULE
USER CONTROLLER
USER SERVICE
CUSTOM CLASS
Decisions
EXPORT?
PROVIDE?
POST MODULE
POST CONTROLLER
POST SERVICE
CUSTOM CLASS

**********


**********

To use services inside other services:

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  // eslint-disable-next-line prettier/prettier
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService], // this service become available to use by other
modules
})
export class UsersModule {}
```

---

```
import { Module } from '@nestjs/common';
import { PostsController } from './posts.controller';
import { PostsService } from './posts.service';
import { UsersModule } from 'src/users/users.module';

@Module({
  controllers: [PostsController],
  providers: [PostsService],
  imports: [UsersModule], // this will import only the services that are exported
in UsersModule
})
export class PostsModule {}
```

---

```
import { Injectable } from '@nestjs/common';
import { UsersService } from 'src/users/users.service';

@Injectable()
export class PostsService {

    constructor(private readonly usersService: UsersService) {

    }

}
```

---

To solve the circular dependency:

```
import { forwardRef, Module } from '@nestjs/common';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { UsersModule } from 'src/users/users.module';

@Module({
  controllers: [AuthController],
  providers: [AuthService],
  imports: [forwardRef(() => UsersModule)],
  exports: [AuthService],
})
export class AuthModule {}
```

---

Then inside the other module (UsersModule):

```
import { Module, forwardRef } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';
import { AuthModule } from 'src/auth/auth.module';

@Module({
  // eslint-disable-next-line prettier/prettier
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService], // this service become available to use by other
modules
  imports: [forwardRef(() => AuthModule)],
})
export class UsersModule {}
```

---

Then inside the services of each module (users, auth services):

```
import { forwardRef, Inject, Injectable } from '@nestjs/common';
import { UsersService } from 'src/users/users.service';

@Injectable()
export class AuthService {

    constructor(
        @Inject(forwardRef(() => UsersService))
        private readonly usersService: UsersService
    ) {}
}
```

---

```
import { forwardRef, Inject, Injectable } from '@nestjs/common';
import { AuthService } from 'src/auth/auth.service';

@Injectable()
export class UsersService {
    constructor(
        @Inject(forwardRef(() => AuthService))
        private readonly authService: AuthService
    ) {}
}
```

**********